

CSE 234 Project 1: RAG and Context Engineering

Owen Park
owpark@ucsd.edu

Shaurya Raswan
sraswan@ucsd.edu

1 Data Creation Methodology

1.1 Line Coverage Analysis

To create our Q&A pairs, we started by identifying gaps in the validation set to better cover the documentation. We wrote a script to measure line coverage for each documentation file and used it to target questions from uncovered lines. The line coverage results for each file, across both the validation and custom datasets, are shown below:

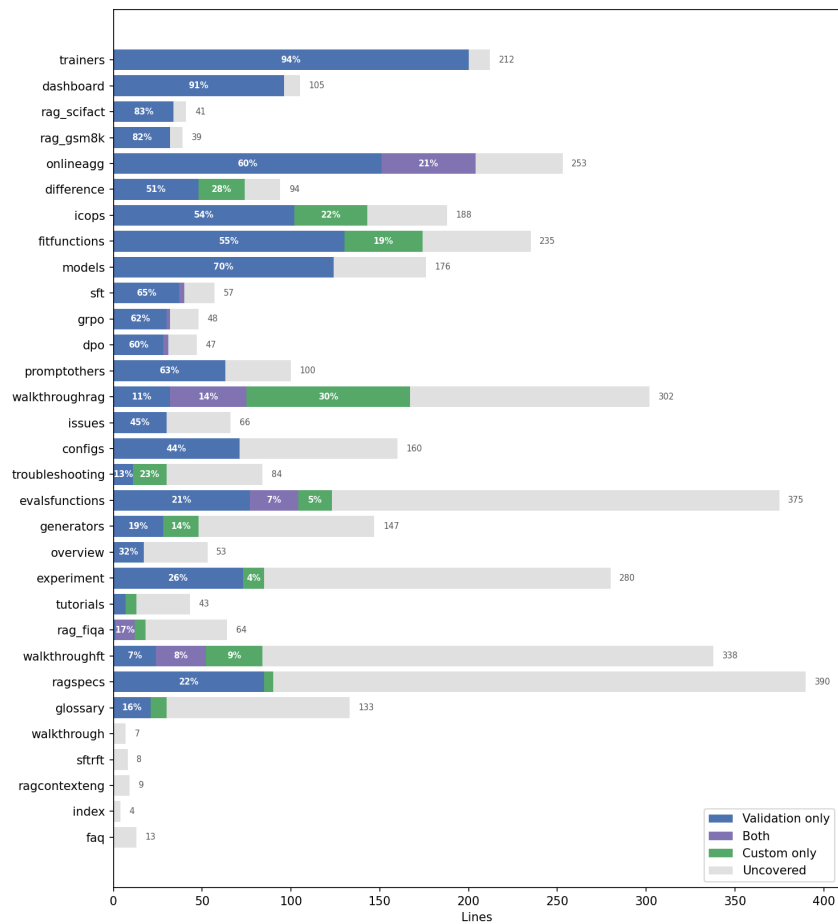


Figure 1: Documentation Line Coverage of Datasets

As the results show, most of our questions covered lines that were missing from the validation set. We intentionally avoided files that were already well-covered, such as `trainers.rst`, `dashboard.rst`, etc.

1.2 Question Type Analysis

In addition to line coverage, we also wanted diversity among the types of questions we asked. To achieve this, we kept the distribution of question types roughly even across both the validation and custom datasets:

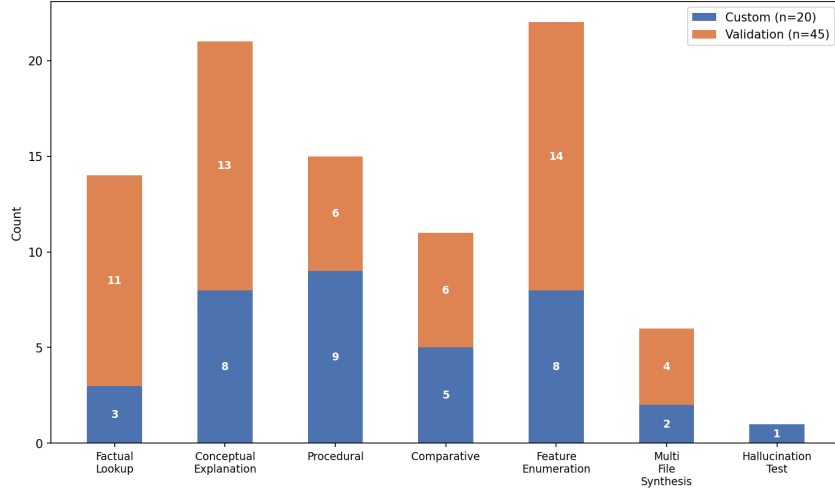


Figure 2: Question Type Totals

As the results show, all question types are well-represented, with the exception of multi-file synthesis and hallucination tests, which is a new category we will talk about later. Multi-file synthesis questions are naturally harder to construct, and hallucination tests are few since they represent an edge case.

1.3 Quality of Questions

After identifying the gaps in line coverage and question type distribution, we wrote Q&A pairs with the LLM judge criteria in mind to make sure each answer was correct, faithful to the retrieved context, and and complete.

1.4 Iteration and Augmentation

After drafting our initial Q&A pairs, we used an LLM to revise them for style and completeness. Since our evaluation relies on an LLM judge, it made sense to align the style and phrasing of our answers with how an LLM would naturally express information. We also revisited the question type distribution after the initial draft and adjusted our set to fill in underrepresented categories.

1.5 Hallucination Test

Since the LLM judge prompt mentions for refusals when a question cannot be answered by the documentation, we wanted to explicitly test this behavior. We added a question to our dataset where the answer is not found in the documentation to see whether the model hallucinates or correctly refuses to answer.

2 Final Pipeline Architecture

The pipeline follows a modular sequence: ingestion, retrieval, dynamic re-ranking, and constrained generation.

2.1 Data Ingestion

The system processes the source documentation through a structured ingestion layer with a standard chunking strategy. We use `RecursiveCharacterTextSplitter` with a chunk size of 300 tokens with an overlap of 64 tokens. We used a custom chunk line map that is precomputed, which simply maps every text chunk back to its specific 1-based line span in the original source files. This ensures high-granularity citations and accurate context reconstruction. This is something that was also required because the `tiktoken` text splitter has a known issue with `add_start_index` being dropped sometimes. Finally, the embeddings are generated using `api-tgpt-embeddings` and stored in a FAISS vector database for efficient similarity search.

To maximize relevance while minimizing noise, the pipeline has 2 stages similar to what was outlined in lecture.

2.2 Initial MMR Retrieval

The system uses Maximal Marginal Relevance (MMR). It gets a diverse set of candidate documents ($k = 10$, $fetch_k = 20$, $lambda_mult = 0.7$), reducing redundancy among the retrieved chunks. The lambda represents the ratio between similarity and diversity when retrieving documents. We saw this perform better than a pure similarity score with a threshold.

2.3 Dynamic Re-ranking

A CrossEncoder model (`BAAI/bge-reranker-large`) rescores on the retrieved MMR candidates. This is not in the `RFLangChainRagSpec` pipeline and we had to load the model ourselves to get score values. First we do contextual filtering with some "score gap" logic. It keeps track of the best-scoring document and retains other chunks only if their scores fall within a specific threshold (within 0.2 for the same file, within 0.01 for different files). This is because the chunks within the same file as the best-ranked chunk are most likely to further support answering the question. In addition, we have a merging helper function we use later on that merges overlapping chunks within the same file so we do not send repeated text as context, and to make our line spans more accurate.

Finally, we sort. The results are re-ordered based on the cross-encoder scores to ensure the most relevant information is prioritized within the token budget

2.4 Context Optimization

Before generation, the retrieved chunks undergo a merging and truncation process as mentioned. We do span merging, so overlapping or contiguous line spans from the same source file are merged into single blocks of text to save tokens and improve readability.

In addition, the system dynamically calculates our max available tokens to send by subtracting the system prompt and question overhead from the total budget. It appends merged chunks until this limit is reached.

2.5 Constrained Generation

The final answer is generated using `api-mistral-small-3.2-2506`. The generation is governed by a strict system prompt that enforces three pillars of quality that the LLM as Judge grades with.

- **Correctness:** Requires leading with a clear, unambiguous main claim.
- **Faithfulness:** Prohibits the use of external knowledge. If the context is insufficient, the model must explicitly state so.
- **Completeness:** Mandates that all sub-questions within the user query are identified and addressed.

Finally, we append our answer, cleaned context, and merged source line spans and file names.

3 Experimentation Methodology

3.1 Initial Incomplete Experimentation

We initially tested a large combination of configurations, with a search space spanning hundreds of configs. We quickly realized how impractical this was, as we ran into issues with API limits and periodic disconnects from Datahub. It became clear that instead of one large experiment, we should conduct several smaller, focused experiments to figure out which parameters actually mattered instead of blindly searching over all combinations.

3.2 8 Distinct Configs

Using preliminary results from the few completed shards, we narrowed our search space down to parameters that seemed to make a meaningful difference, despite the small sample size. Some of these parameters included chunk size, retrieval strategies such as similarity score threshold and MMR, and our custom-defined parameters such as the score gap threshold between same-document and different-document results. In this particular experiment with 8 configs, we focused on chunk size, chunk overlap, embedding model, and retrieval strategy:

ID	CS	CO	Embeds	Search Config
1	300	64	api-tgpt	Sim (k=7, t=0.4)
2	300	64	api-tgpt	MMR (k=10, f=20)
3	300	64	mxbai-large	Sim (k=7, t=0.4)
4	300	64	mxbai-large	MMR (k=10, f=20)
5	512	96	api-tgpt	Sim (k=7, t=0.4)
6	512	96	api-tgpt	MMR (k=10, f=20)
7	512	96	mxbai-large	Sim (k=7, t=0.4)
8	512	96	mxbai-large	MMR (k=10, f=20)

Table 1: RAG configurations tested. **CS**: Chunk Size, **CO**: Chunk Overlap.

3.3 Further Experimentation

After our 8 main configs, we focused on parameters we had not yet explored. We had intentionally left out our custom-defined dynamic re-ranking parameters, described in 2.2, to keep the search space from blowing up. With the main parameters settled, we were able to experiment with these on their own. The implementation in RapidFire AI was a bit of a challenge as we had to create a wrapper to modify the reprocess function and the compute metrics functions. Nonetheless, here are the configurations we tried:

ID	Same	Other	Lambda
1	0.2	0.01	0.7
2	0.2	0.06	0.7
3	0.3	0.01	0.8
4	0.3	0.01	0.7
5	0.2	0.01	0.8

Table 2: Configurations tested. **Same**: Same-document score gap threshold, **Other**: Different-document score gap threshold, **Lambda**: MMR lambda parameter.

4 Results and Analysis

4.1 Fixed Configuration Parameters

For all experimental runs (1–8), the following parameters remained constant:

- **Model Name**: api-mistral-small-3.2-2506
- **Cross-Encoder**: BAAI/bge-reranker-large (Top N=4)

ID	CS	CO	Embeds	Search Config	F1@5	Prec	Rec	Retr	Corr	Faith	Comp	Gen
1	300	64	api-tgpt	Sim (k=7, t=0.4)	0.764	0.735	0.894	0.798	0.978	0.978	0.8	0.919
2	300	64	api-tgpt	MMR (k=10, f=20)	0.767	0.720	0.894	0.809	1.000	1.000	0.818	0.939
3	300	64	mxbai-large	Sim (k=7, t=0.4)	0.759	0.726	0.894	0.793	0.956	0.978	0.809	0.914
4	300	64	mxbai-large	MMR (k=10, f=20)	0.767	0.737	0.894	0.799	0.956	0.978	0.8	0.911
5	512	96	api-tgpt	Sim (k=7, t=0.4)	0.744	0.733	0.85	0.776	0.956	0.978	0.876	0.936
6	512	96	api-tgpt	MMR (k=10, f=20)	0.737	0.711	0.872	0.773	0.956	0.978	0.849	0.927
7	512	96	mxbai-large	Sim (k=7, t=0.4)	0.748	0.722	0.894	0.788	0.956	1.000	0.836	0.930
8	512	96	mxbai-large	MMR (k=10, f=20)	0.744	0.719	0.894	0.786	0.978	1.000	0.849	0.942

Table 3: Comparative performance of RAG configurations. **CS**: Chunk Size, **CO**: Chunk Overlap, **Retr**: Retrieval, **Faith**: Faithfulness, **Corr**: Correctness, **Gen**: Generation. **Prec**: Precision. **Rec**: Recall. **Comp**: Completeness.

- **Score Gaps**: Same File = 0.35, Other File = 0.1
- **Vector Store**: FAISS (Batch Size = 32)
- **Temperature**: 0.0
- **Tokenizer**: RecursiveCharacterTextSplitter
- **LLM as Judge**: api-mistral-small-3.2-2506. This is probably why the generation score is so high.

Above is the comparative performance of 8 essential RAG configurations we used to hone in on the best knobs. We later ran individual tests to find the best thresholds next. This also allows us to use higher cost judge models.

We clearly saw smaller chunk sizes (at around 300 to 256) outperform 512 chunk sizes in the retrieval score. An overlap of around 20% of our chunk size also performed best. This was surprising as we believed larger chunk sizes would contain more context overall, leading to higher recall. Although recall was similar, precision and F1 were generally lower when using bigger chunk sizes. Due to the increased context, the generation score generally did better for larger chunk sizes. We can also see this by completeness scores, which are much worse for 300 chunk size.

We also saw api-tgpt do worse in retrieval in the larger chunk sizes, but do similarly to mxbai-large in the smaller ones. This is possibly because mxbai-large is probably specialised for more standard chunk sizes such as 512. Surprisingly, MMR did not impact our results as much as we thought it would, but maybe we needed to decrease the lamdba to see a difference.

After the leaderboard scores, we used the parameters below:

```
LLM_MODEL = "api-mistral-small-3.2-2506"
CHUNK_SIZE = 300
CHUNK_OVERLAP = 64
SEARCH_CONFIG = "similarity"
RETRIEVAL_K = 7
RERANK_MODEL = "BAAI/bge-reranker-base"
RERANK_TOP_N = 4
RERANK_SCORE_GAP_SAME = 0.35
RERANK_SCORE_GAP_OTHER = 0.1
```

And we achieved 10th place, with

Leaderboard	F1	Prec	Rec	Retr	Corr	Faith	Comp	Hidden1	Hidden2	Gen
0.76995	0.71545	0.68889	0.85	0.75145	0.71111	0.68889	3.71111	0.97778	0.82222	0.78844

Table 4: Leaderboard scores.

This inspired us to improve our retrieval score after seeing that all of our metrics were underperforming. Therefore, we tried the following hyperparameters for thresholding:

ID	Same	Other	Lambda	F1@5	Prec	Rec	Retr	Corr	Faith	Comp	Gen
9	0.3	0.01	0.7	0.8443	0.8259	0.9389	0.8697	0.84	0.87	0.728	0.8133
10	0.3	0.01	0.8	0.8073	0.7926	0.8944	0.8314	0.91	0.91	0.778	0.8667
11	0.2	0.01	0.7	0.8583	0.8444	0.9389	0.8805	0.89	0.89	0.724	0.8341
12	0.2	0.01	0.8	0.8213	0.8111	0.8944	0.8423	0.84	0.91	0.728	0.8281
13	0.2	0.06	0.7	0.7968	0.7611	0.9389	0.8323	0.78	0.91	0.702	0.7970
14	0.2	0.01	0.6	0.8368	0.8333	0.8944	0.8549	0.87	0.93	0.694	0.8311

Table 5: Comparative performance of RAG configurations. **Same**: Threshold for files the same as the best ranked, **Other**: Threshold for files other than the best ranked, **Lambda**: Lambda mult. All parameters are the same now, with LLM as judge as `api-gpt-oss-120b`.

As we can see, the new judge model we are using is stricter than previously. In addition, it is stricter than Claude Sonnet based on empirical tests. Our retrieval is significantly better after making our same and other thresholds stricter. We observed that being stricter on our threshold for including context that is outside of the best-ranked file did worse in both retrieval and generation.

The lambda has a trade-off of precision vs. faithfulness. There is a clear inverse relationship between retrieval metrics and generation quality when adjusting the MMR Lambda. A Lower Lambda (0.7) has higher F1, precision, and retrieval Scores. This is possibly due to how more diversity helps surface the correct documents. Higher Lambda has retrieval scores drop, but faithfulness and generation scores generally improve. This indicates that a more relevant-heavy search provides a cleaner context for the LLM, even if it misses some marginal documents.

The same file score gap (the threshold for keeping chunks from the same document as the top result) significantly impacts retrieval success. A tighter gap (0.2) is superior to a wider gap (0.3). A tighter threshold likely filters out filler chunks from the same document that might otherwise dilute the context, allowing the LLM to focus on higher-density information.

These tradeoffs make it hard to pick the best hyperparameters, as run 10 and 11 specialize in either retrieval or generation score.

Therefore, our final chosen hyperparameters are:

```
LLM_MODEL = "api-mistral-small-3.2-2506"
CHUNK_SIZE = 300
CHUNK_OVERLAP = 64
SEARCH_CONFIG = "mmr"
RETRIEVAL_K = 10
FETCH_K = 20
LAMBDA_MULT = 0.7
RERANK_TOP_N = 4
RERANK_MODEL = "BAAI/bge-reranker-large"
RERANK_SCORE_GAP_SAME = 0.2
RERANK_SCORE_GAP_OTHER = 0.01
```

4.2 Future Testing

We would like to test more by comparing architectural differences, not just hyperparameters, such as how Hypothetical Document Embeddings (HyDE) might compare with our current architecture, or how sorting based on cross encoding might be different from not sorting. One of the main architecture differences we wished we could test is how MMR impacts document retrieval thresholds with our "score gap" methodology. Decreasing lambda might lead to situations where we get from a very diverse set of different files entirely, which might not cause good results with our thresholding that incentivizes picking chunks within the same file. Instead, if we had a new pipeline that uses low lambda MMR, it might expose files that are truly the best ranking if the same file appears in multiple received chunks. So instead of thresholding by score, we could also threshold by the number of appearing chunks within the same file.

In addition, we would like to run on more datasets (including our own golden qa pairs). We also would have liked to create our own metrics in the validation set and for the judge to grade us on.

5 AI Tools Statement

For our implementation, we used LLM tools such as Claude and ChatGPT to help generate code that was mostly boilerplate and graphing. We also consulted LLMs to better understand modern RAG techniques and to get ideas on possible parameters to explore. In all cases, we made sure we understood the code and made sure that it was correct before using it.